

```

        printf("Divyajyoti");
    fun (n - 1); ← T (n - 1)
}
}

```

Solution. $T(n) = T(n - 1) + O(\log_2 n); \quad n > 0$

or

$$T(n) = T(n - 1) + \log_2 n$$

$$T(0) = C \quad \left. \begin{array}{l} \\ \end{array} \right\} \text{ or } n = 0$$

$$T(0) = O(1)$$

1.9 Solving Recurrence Relation

1.9.1 Substitution Method

Example: (1)

$$T(n) = T(n - 1) + C$$

$$T(1) = C$$

n size on problem $n - 1$ size x_1 convert them

$$T(n) = T(n - 1) + C$$



$$[T(n - 2) + C] + C$$

$$T(n) = T(n - 2) + 2C$$



$$= T(n - 3) + 3C$$

$$T(n) = T(n - k) + kC$$

$$\therefore n - k = 1$$

$$T(n) = T(1) + (n - 1)C$$

$$= C + (n - 1)C$$

$$T(n) = O(n)$$

Example (2)

$$T(n) = T(n - 1) + C \cdot n$$

$$T(1) = C$$

Solution.

$$\therefore T(n) = T(n - 1) + C \cdot n$$

$$= [T(n - 2) + C \cdot (n - 1)] + C \cdot n$$

$$= [T(n - 3) + C \cdot (n - 2)] + C \cdot (n - 1) + C \cdot n$$

$$= T(n - 3) + (n - 2) \cdot C + (n - 1) \cdot C + n \cdot C$$

$$= T(n - k) + C(n - k + 1) + C(n - k + 2) + \dots + C(n - k + k)$$

$$\therefore n - k = 1$$

$$T(n) = T(1) + T(2) + C(3) + C(4) + \dots + C(n - 1) + C(n)$$

$$= C + C(2) + (3)C + 4(C) + \dots + (n - 1)C + (n) \cdot C$$

$$= C[1 + 2 + 3 + \dots + n]$$

$$= C \cdot n \frac{(n+1)}{2}$$

$$= O(n^2)$$

Example (3)

$$T(n) = T(n/2) + C$$

$$T(1) = 1$$

Solution.

$$T(n) = T(n/2) + C$$

$$= [T(n/2^2) + C] + C$$

$$= T(n/4) + 2C$$

$$= T(n/2^3) + 3C$$

$$T(n) = T(n/2^k) + kC$$

$$= (n/2^k) = 2$$

$$T(n) = T(2) + (\log_2 n - 1) C$$

$$= 1 + (\log_2 n - 1) C$$

$$= O(\log n)$$

Example (4)

$$T(1) = 1$$

$$T(n) = 2T(n/2) + C$$

Solution. $T(n) = 2 \left[2T\left(\frac{n}{2^2}\right) + C \right] + C$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2^2 C + C$$

$$= 2^2 \left[2T\left(\frac{n}{2^3}\right) + C \right] + 2C + C$$

$$= 2^3 T\left(\frac{n}{2^3}\right) + 2^2 C + 2C + C$$

$$= 2^k T\left(\frac{n}{2^k}\right) + 2^{k-1} C + 2^{k-2} C + \dots + 2^1 \cdot C + C$$

$$\frac{n}{2^k} = 1 \quad \therefore n = 2^k$$

$$\begin{aligned} \rightarrow T(n) &= nT(1) + 2^{k-1} \cdot C + 2^{k-2} \cdot C + \dots + 2C + C \\ &= 2^k + 2^{k-1} \cdot C + 2^{k-2} \cdot C + \dots + 2C + C \\ &= 2^k + C(2^{k-1} + 2^{k-2} + \dots + 2^1 + 2^0) \\ &= 2^k + C \frac{(2^k - 1)}{2 - 1} \\ &= 2^k + C(2^k - 1) \\ &= 2^k + 2^k \cdot C - C \\ &= n \cdot C \\ &= O(n) \end{aligned}$$

1.9.2 Master's Method

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n^k(\log n)^p)$$

$a \geq 1, b > 1, k \geq 0, p = \text{real number}$

If $a > b^k$ or $\log_b a > k$
 $T(n) = \Theta(n^{\log_b a})$

Question 1.

$$T(n) = 2T\left(\frac{n}{2}\right) + (n)^0 \log n$$

Solution.

$$\begin{aligned} a &= 2, b = 2, k = 0 \\ a &> b^k; 2 > 2^0; 2 > 1 \\ T(n) &= \Theta(n) \end{aligned}$$

Question 2.

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Solution.

$$\begin{aligned} a &= 2, b = 2, k = 1, p = 0 \\ T(n) &= \Theta(n \cdot \log n) \end{aligned}$$

Question 3.

$$T(n) = 2T\left(\frac{n}{2}\right) + n \log n$$

Solution.

$$\begin{aligned} a &= 2, b = 2, k = 1, p = 1 \\ \therefore T(n) &= \Theta(n (\log n)^2) \end{aligned}$$

(b) If $p < 0$ then $T(n)$
 $T(n) = O(n^k)$

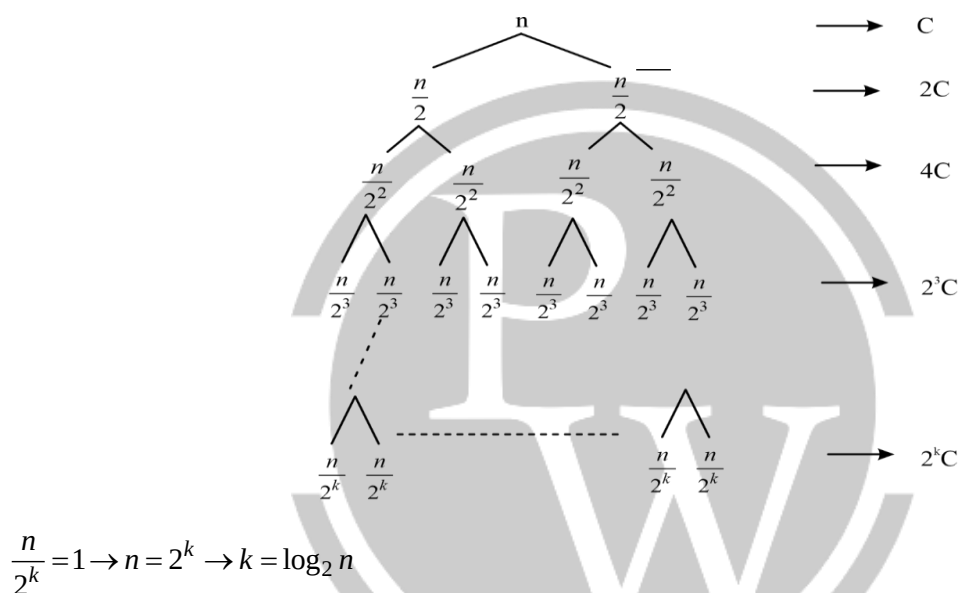
Question 4. $T(n) = T\left(\frac{n}{2}\right) + C$

Solution.

$$T(n) = \Theta(n^2 \log n)$$

1.9.3. Recursive Tree

(1) $T(n) = 2T\left(\frac{n}{2}\right) + C$
 $T(1) = C$



$$\text{Total Work done} = C + 2C + 2^2C + 2^3C + \dots + 2^kC$$

$$= C (1 + 2 + 2^2 + \dots + 2^k)$$

$$= C \left(\frac{2^{k+1} - 1}{2 - 1} \right)$$

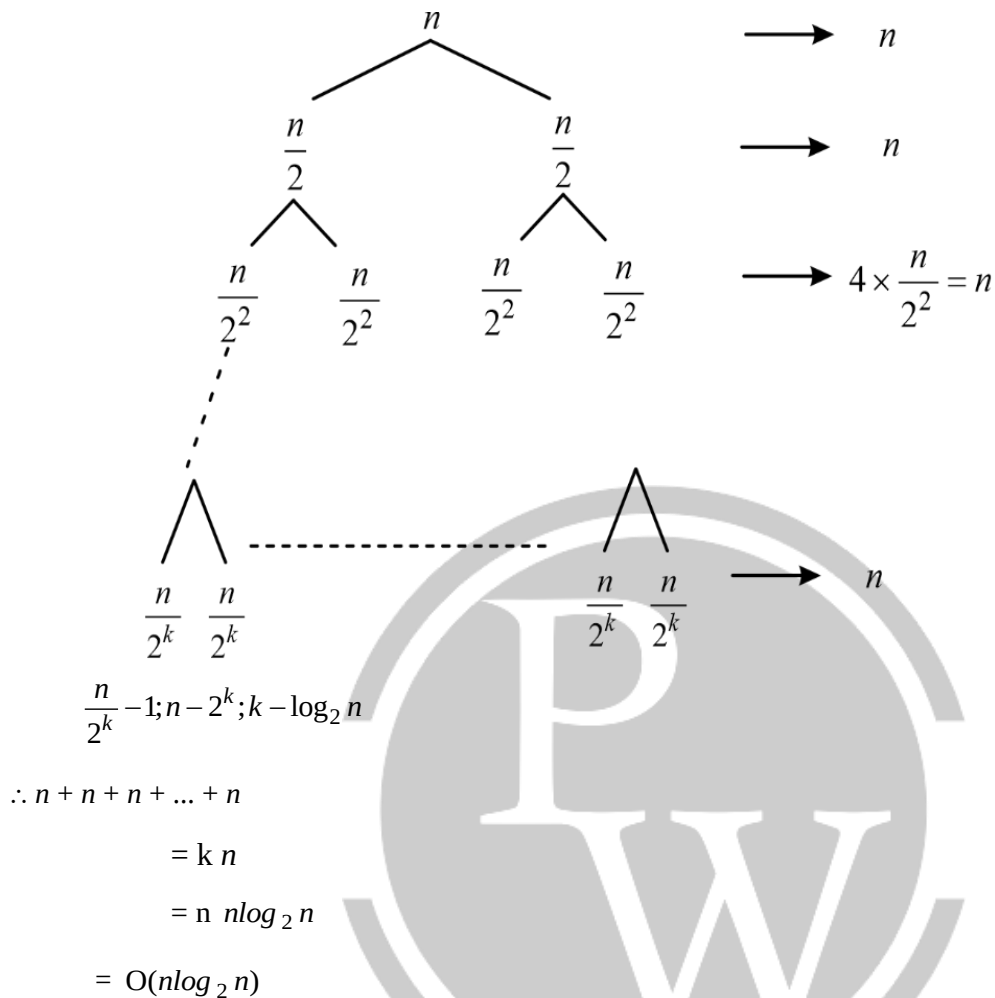
$$= C (2^{k+1} - 1)$$

$$= C (2 \cdot 2^k - 1)$$

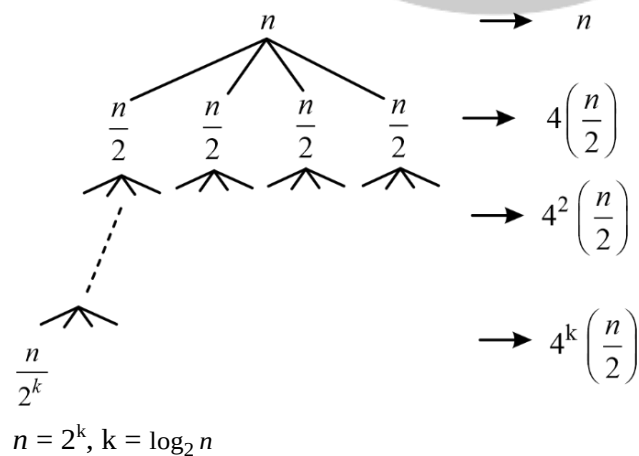
$$= C (2n - 1)$$

$$= O(n)$$

(2) $T(n) = 2T\left(\frac{n}{2}\right) + n$



(3) $T(n) = 4T\left(\frac{n}{2}\right) + n$



$$= n + 4\left(\frac{n}{2}\right) + 4^2\left(\frac{n}{2}\right) + \dots + 4^k\left(\frac{n}{2}\right)$$

$$= n[1+2+2^2+2^3+\dots+2^k]$$

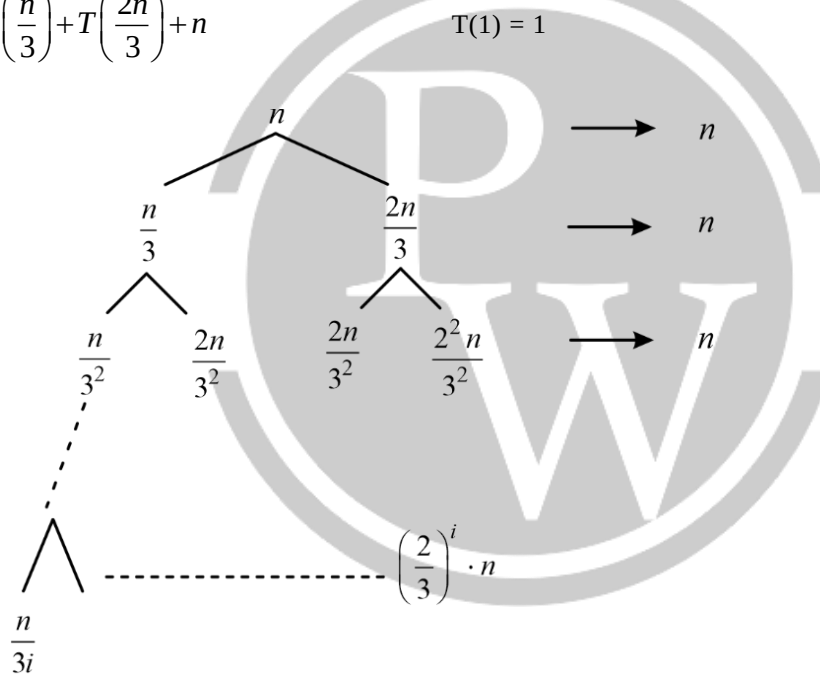
$$= n\left(\frac{2^{k+1}-1}{2-1}\right)$$

$$= n(2 \cdot 2^{k-1})$$

$$= n(2n) - 1$$

$$= O(n^2)$$

(4) $T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$



$$\frac{n}{3^i} - 1; n - 3i; i - \log_3 n$$

$$= n + n + \dots + \log_3 n \cdot T(n)$$

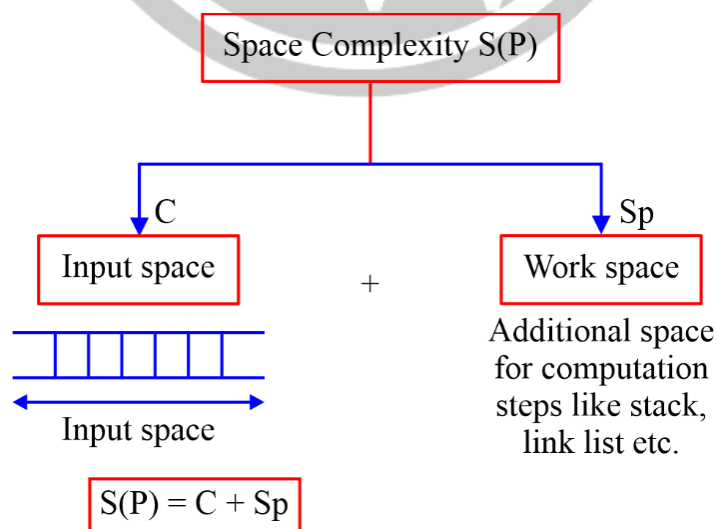
$$= (n + n + n + \dots + \log_3 n) \geq n + \dots + \log_3 n$$

$$\Omega(n \log_{3/2} n)$$

1.10 Recurrence Relations and their Time Complexity

$T(n) = C; n = 2$ $T(n) = 2T(\sqrt{n}) + C; n > 2$	$O(\log n)$
$T(n) = C; n = 2$ $T(n) = T(n-1) + C; n > 2$	$O(n)$
$T(n) = C; n = 1$ $T(n) = T(n-1) + n + C; n > 2$	$O(n^2)$
$T(n) = C; n = 1$ $T(n) = 2T(n-1) + C; n > 1$	$O(2^n)$
$T(n) = C; n = 1$ $T(n) = 2T\left(\frac{n}{2}\right) + C; n > 1$	$\theta(n)$
$T(n) = C; n = 1$ $T(n) = 2T\left(\frac{n}{2}\right) + n; n > 1$	$\theta(n \log n)$
$T(n) = C; n = 1$ $T(n) = T\left(\frac{n}{2}\right) + C; n > 1$	$\theta(\log n)$
$T(n) = 1; n = 2$ $T(n) = T(\sqrt{n}) + C; n > 2$	$\theta(\log \log n)$

1.11 Space Complexities



```

Int n, A[n];
Algorithm Rsum(A, n)
{
    if (n = 1) return (A(1));
    else;
    return (A[n] + RSum(A, (n-1)));
}

```

- **Time Complexity** = $O(n)$
- **Space Complexity**
- We need stack space
- Stack is used to store activation records of function calls
- Size of activation records is trivial
- Stack size that we need = $O(n)$
- Space complexity = $O(n)$

```

Algorithm A(n)
{
    if (n = 1) return;
    else;
    {
        A( $\frac{n}{2}$ );
    }
}

```

Recurrence relation

$$T(n) = C; n = 1$$

$$T(n) = T\left(\frac{n}{2}\right) + C; n > 1$$

$$\text{Time Complexity} = O(\log n)$$

Space Complexity

- Space complexity will depend on number of activation record pushed into the stack
Suppose, $n = 16$

A (1)
A (2)
A (4)
A (8)
A (16)

For $n = 2^K$ we are pushing the 'K' activation record

∴ Space Complexity

$$n = 2^K$$

$$\log n = K \log_2 2$$

$$K = \log_2 n$$

$$\boxed{\text{Space Complexity} = O(\log n)}$$

Example 3

```

Algorithm A(n)
{
    if (n = 2) return;
    else;
    return (A  $\sqrt{n}$ );
}
    
```

Solution:

$$T(n) = 1; n = 2$$

$$T(n) = T(\sqrt{n}) + C; n > 2$$

$$\text{Time Complexity} = O(\log \log n)$$

Space Complexity

Suppose $n = 16$

A(1)
A(2)
A(4)
A(16)

∴ For $2^{n/2^k}$ manner we are pushing in stack

$$2^{n/2^k} \geq 2$$

$$\frac{n}{2^k} \log_2 2 \geq \log_2 2$$

$$n \geq 2^K$$

$$K \leq \log_2 n$$

$$\boxed{\text{Space complexity} = O(\log_2 n)}$$

□□□